CS (Computer Science) 251 — Integrative Project 1 (3 cr) · 2026–2027 · Class 2

The Problem Before the Code

Requirements and the research process

Friday, October 2, 2026 · Phase 1 – Analysis · 1 h lecture · 2 h workshop · 2 h at home

Asen Grozdanov (Cyber) · Cyber.SoHo Educational Hub · From the Industry to the Classroom

Today's five lessons

1. **Research problem**
2. **Requirements**
3. **Scope & objectives**
4. **Sources & citation**
5. **Data dictionary**

Use of these materials is subject to the Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy (v2.0).

Today's hour — five lessons, five decisions

1 **0:00–0:11** — **Lesson 1 — From a business problem to a research problem**

Decides **what we are trying to find out** · then: the problem tree (0:11–0:15)

2 **0:15–0:26** — **Lesson 2 — Requirements analysis**

Decides **what the software must do** · then: one **FR** (Functional Requirement) + one **NFR** (Non-Functional Requirement), swap and check (0:26–0:30)

3 **0:30–0:41** — **Lesson 3 — Scope, SMART (Specific, Measurable, Achievable, Relevant, Time-bound) objectives and project selection**

Decides **how we will know it is finished** · then: fun quiz + zen break (0:41–0:45)

4 **0:45–0:50** — **Lesson 4 — References and experts**

Decides **where our evidence comes from**

5 **0:50–0:56** — **Lesson 5 — The data dictionary and personal data**

Decides **what every column of our data means** · then: flash-cards (0:56–1:00)

W **1:00–3:00** — **Workshop — Brick 2 “Requirements & Research Proposal v1”**

Checkpoint at 2:45 · due Thursday, October 8, 23:59

By the end of this class you can…

1. Write a problem statement, three research questions and five user stories for JARVIS.
2. Tell a credible source from a weak one and cite it in **APA** (American Psychological Association) 7.
3. Produce a data dictionary and flag at least three data-quality risks and one privacy risk.



1

Lesson 1 of 5 · lecture 0:00–0:11 · then the problem-tree visualization 0:11–0:15

From a business problem to a research problem

In this lesson

- Opening and the Parking Lot · 0:00–0:02
- From a complaint to a research problem · 0:02–0:05
- Sub-problems · 0:05–0:08
- Research questions · 0:08–0:11



From a complaint to a research problem

Research

A systematic process of **collecting, analyzing and interpreting data** in order to increase our understanding of a phenomenon (Leedy & Ormrod, 2019).

Research problem

A question or a situation that can be resolved **only by collecting data and interpreting it** — the starting point of every research project.

The owner's e-mail (Monday)

"Mornings disappear into the mailbox, the daily report is always late, and a vendor wants 20 $ per person per month for an assistant. Before I sign anything, I want to know what to automate first."

That is a **complaint**: it does not say…

- ✗ what will be **measured**
- ✗ what **decision** the answer will support
- ✗ **how** we will reach the answer

A problem statement turns the complaint into one short paragraph that names three things: the situation, the decision to be made, and the evidence that will be collected.



The problem statement — situation, decision, evidence

Situation

Boréal Studio, a five-person Montréal consultancy, spends a large part of every working day on six recurring routines.

Decision

The owner must decide which routine to automate first, and whether a free assistant is sufficient or a paid tier at 20 $ per user per month is justified.

Evidence

This project measures how long each routine takes, prices that time at 28 $ per hour, builds a Python assistant that automates at least one routine, and compares the time it saves with the cost of each option, in order to recommend what to automate first.

93 words

Brick 2 allows at most **150 words**.

Leedy & Ormrod's guidance

- State it **clearly and completely**.
- Make it **feasible**: fifteen Fridays, a team of two or three, free tools.
- **Define the words** that could be read in two ways.
- Keep an **open mind**: no conclusion written into the problem.



What is not a research problem

✗ **Only informing yourself**

"Learn about chatbots" — a task whose only purpose is self-enlightenment.

✗ **Comparing without interpreting**

Putting two sets of data side by side and stopping there.

✗ **A correlation for its own sake**

Computing a correlation only to show that one exists.

✗ **A yes-or-no question**

"Is automation good?" — answered without interpreting any data.

*In each case nothing has to be interpreted, so **no new understanding** is produced.*

✓ **Our problem is a research problem:** deciding which routine to automate first, and whether a paid tier is worth 20 $ per user per month, requires **measuring and interpreting** time and cost data.

Sub-problems — four properties, three branches

Good sub-problems (Leedy & Ormrod)

1. Each one is a **researchable unit** by itself.
2. Each one is **tied to the interpretation of data**.
3. Together they **add up to the whole problem** — no more, no less.
4. There are **few** of them: usually two to six.

Problem: which routine to automate first — and is a free assistant good enough?

Microeconomics

What does each routine cost per month, and what does each option cost: free, paid tier, nothing?

Statistics

How long does each routine take, how much does it vary, and did anything change after day 8 and day 10?

Python and OOP (Object-Oriented Programming)

How can an assistant run a routine and measure the minutes it saves?

✗ **Pseudo-sub-problems** — “Which chart library do we use?”, “How do we split the work?”: planning decisions that must be made, but answering them interprets no data, so they stay **out of the problem tree**.

Research questions (1/2) — the course template

A **research question** is a sub-problem written as an **open question** that the data and the methods of the project can answer. Each team refines this template in Brick 2.

Research question	Course	Data that answers it	Method	Answered in
RQ1 (Research Question 1) — Which routines cost the most time, and which are the most variable?	Statistics	routine, minutes, staff_id, date	mean, median, **SD** (standard deviation) per routine; outliers	Brick 4 (Oct 22); Brick 11
RQ2 (Research Question 2) — What is an hour of staff time worth, what does automation cost, and where is the break-even between the free tier and a paid assistant?	Microeconomics	minutes, routine + 28 $/h, 20 $/user/month, free quota	monthly cost, opportunity cost, break-even hours	Brick 3 (Oct 15); Brick 11
RQ3 (Research Question 3) — Which routines should JARVIS automate first, how much time does it save, and what is the risk?	Python (integration)	tool, minutes, date (before / after day 10)	before/after comparison; recommendation pattern	Brick 7 (Nov 12); Brick 11; Final

In January the Results section of your report must answer exactly these questions — and nothing else.



Research questions (2/2) — four checks before you accept one

1 Open question

It cannot be answered by yes or no.

2 Measurable

It names minutes, interruptions, runs or dollars.

3 One sub-problem

It belongs to exactly one branch of the tree.

4 Answerable by January 22

Your data and methods can answer it in time.

✗ **“How can the studio become more productive?”** — fails check 2: *productive* has no unit in our data.

✗ **“Is automation good?”** — fails checks 1 and 2: yes/no, and nothing measurable.

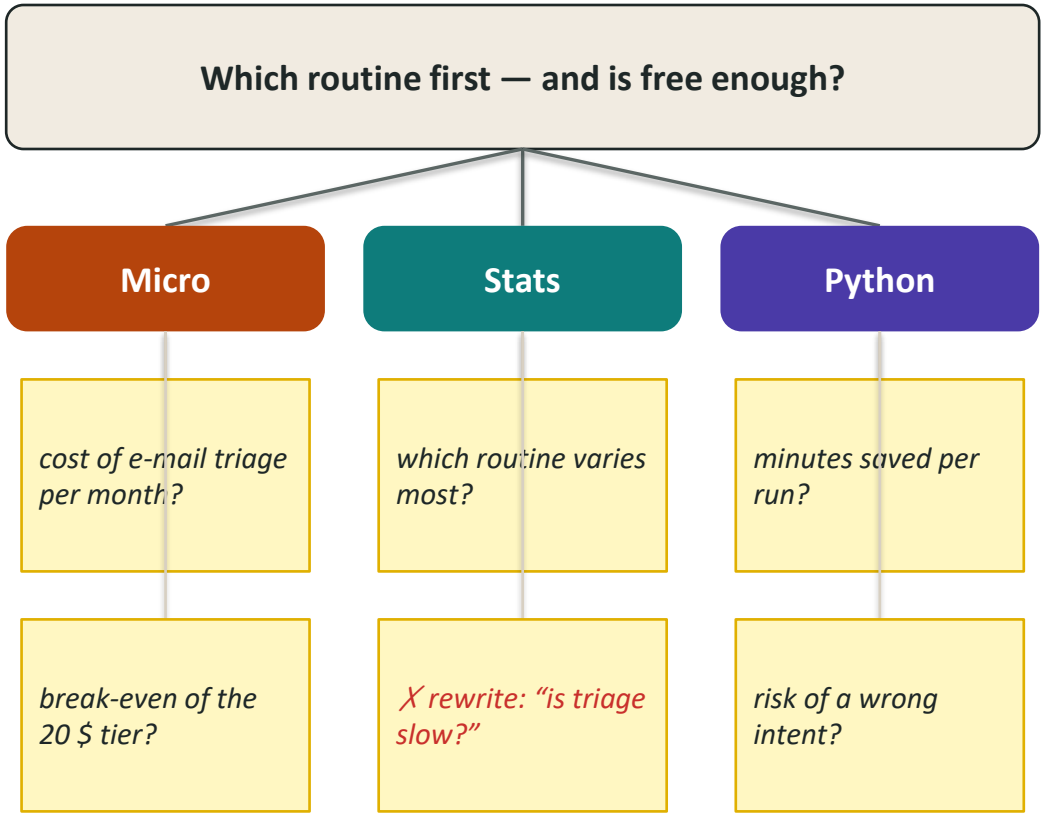
Why spend ten minutes on this?

A vague research question written today produces a vague conclusion in January. Part B of the **Mid-Term** (November 20) gives you a new small-business scenario and asks for a problem, one sub-problem and one research question.

Activity — the problem tree

4 min

- On the board: the owner's question at the top, **three branches** below it: Microeconomics, Statistics, Python.
- Each student adds **one leaf**: a sub-question under one branch.
- The class applies the **four checks** to every leaf: open? measurable? one sub-problem? answerable by January 22?
- A leaf that fails is **rewritten on the spot**, not erased.

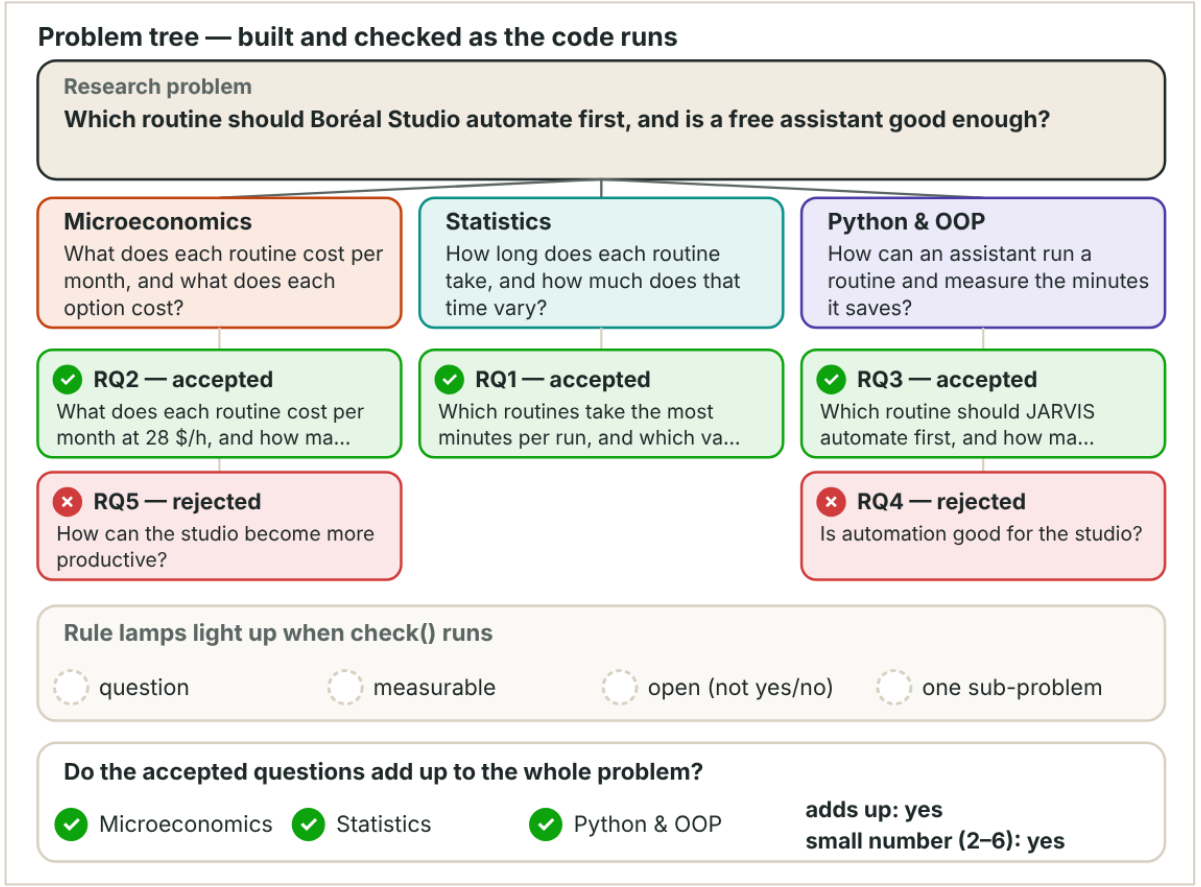


Real-life example — the owner's e-mail becomes a problem tree

1. **Problem statement** — 93 words, checked against Leedy & Ormrod's guidance.
2. **Sub-problems** — cost (Micro), time and variability (Stats), automation (Python).
3. **Research questions** — five drafted; three pass the four checks and become **RQ1** (Research Question 1)–**RQ3** (Research Question 3); two are rejected.
4. **Data map** — for each **RQ** (Research Question): the columns, the method and the Brick that answers it.

Where it goes next

Brick 2 (Oct 8): problem statement + **RQ1**–**RQ3** → report §1. Bricks 3, 4 and 7 answer **RQ2** (Research Question 2), **RQ1** and **RQ3**; **Brick 11** answers all three with numbers and tables. **Mid-Term** Part B: the same steps on a new scenario.



Demo 1 — problem tree and research-question checker. Real Python, replayed step by step in the lecture module: Start / Pause, Next step, Restart. **RQ4** (Research Question 4) and **RQ5** (Research Question 5) fail the rules; the accepted questions cover all three branches.

Figure 1 — the problem tree of Boréal Studio

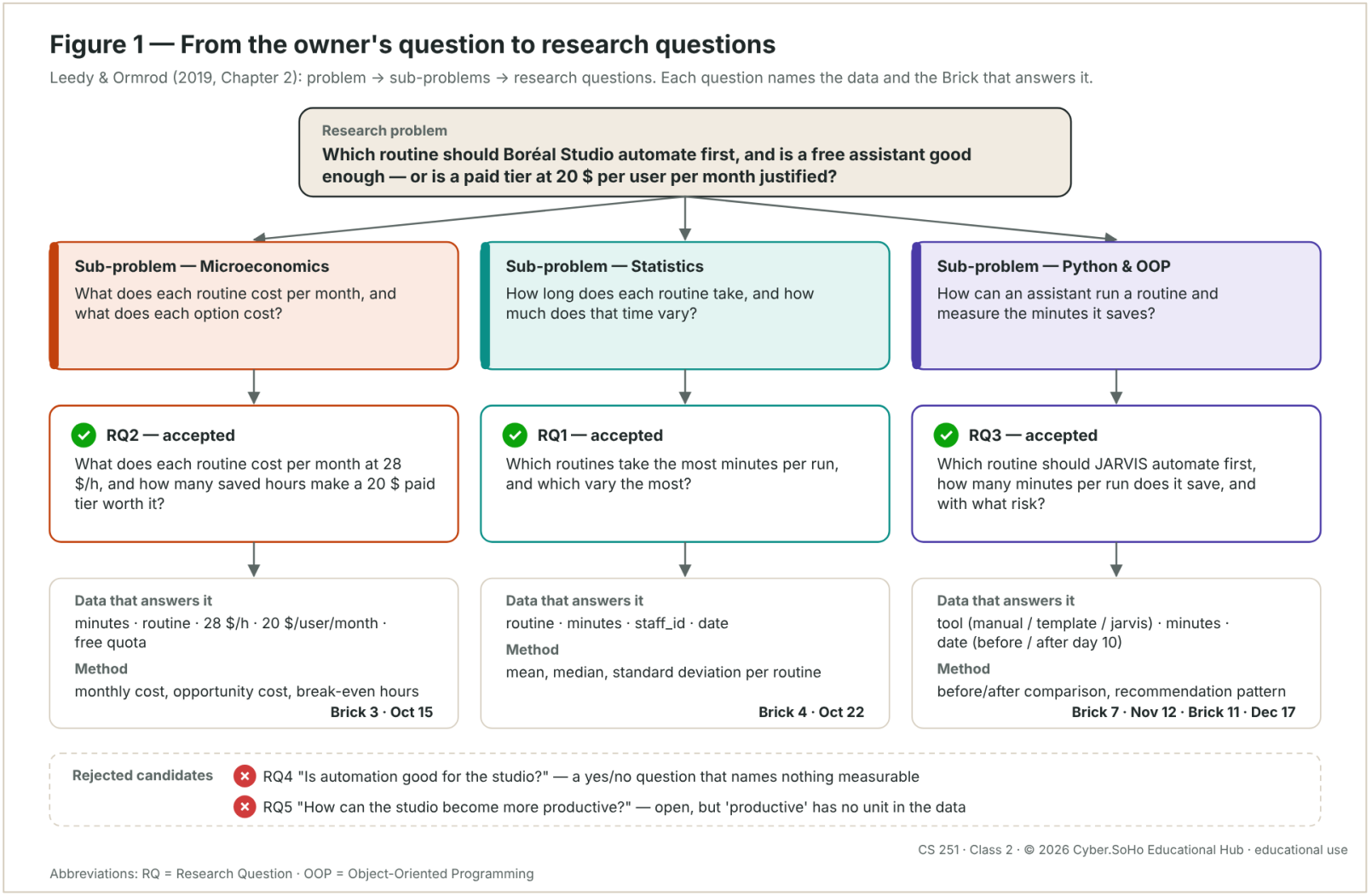


Figure 1 — The problem tree of Boréal Studio: one problem, three sub-problems (one per course), three accepted research questions, and the data and Brick that answer each one.

How to read it

- Top: the owner's problem.
- Middle: three sub-problems, one per course.
- Below each one: the accepted research question, the data that answers it and the Brick.
- Bottom: the two rejected candidates and why they fail.

Remember — Lesson 1

1 **What is a research problem?**

A question or situation that can be resolved only by collecting and interpreting data. It is the starting point of every research project (Leedy & Ormrod, 2019).

2 **What three things does a problem statement name?**

The situation, the decision the research must support, and the evidence that will be collected — in at most 150 words for Brick 2.

3 **Four properties of good sub-problems?**

Each is a researchable unit; each is tied to the interpretation of data; together they add up to the whole problem; they are few (two to six).

4 **Why is "Is automation good?" not a research question?**

It can be answered yes or no, and it names nothing measurable, so it does not say which data to collect or how to interpret them.

5 **What is a pseudo-sub-problem?**

A planning or procedural decision — for example, which chart library to use — that must be made but does not require interpreting data.

▶ **Watch at home:** How to Develop a STRONG Research Question — Scribbr · Picking Your Topic Is Research! — NC State University Libraries, 3:10

2

Lesson 2 of 5 · lecture 0:15–0:26 · then a short practical 0:26–0:30

Requirements analysis

In this lesson

- What a requirement is · 0:15–0:17
- Functional and non-functional requirements · 0:17–0:20
- User stories · 0:20–0:23
- Acceptance criteria · 0:23–0:26



What a requirement is

Requirement

A statement of something a system **must do**, or of a **property** it must have, written so that someone can **check** whether the delivered system satisfies it.

Requirements analysis: discovering, writing, classifying, checking and agreeing on requirements with the **stakeholders**.

Stakeholders of JARVIS

- **the owner**
- **the staff**
- **the Product Defense assessors**

ISO/IEC/IEEE (International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers) 29148:2018 — characteristics of a well-formed requirement

necessary	appropriate	**unambiguous**
complete	**singular**	feasible
verifiable	correct	conforming

Today we use three: **unambiguous** (one reading), **singular** (one requirement per statement), **verifiable** (a test, an inspection or a measurement proves it).

“shall” marks a **binding** requirement; **“should”** marks a goal or a wish. You will use this convention in Brick 2.

Functional versus non-functional requirements

Functional requirement (FR) — what it does

A **behaviour**: given an input or an event, the system produces an output or changes its data.

"The assistant shall log a routine from one command."

"The assistant shall compute the break-even hours for the paid tier."

Non-functional requirement (NFR) — how well, under which limits

A **quality** of the system, or a **constraint** on how it performs its functions.

ISO/IEC (International Organization for Standardization / International Electrotechnical Commission) 25010:2023: nine quality characteristics; ours: performance efficiency, reliability, security, maintainability.

JARVIS has four non-functional requirements, written into the course:

1. runs from a fresh clone in **under 2 minutes**
2. works with **no API (Application Programming Interface) key** and no network
3. **never crashes** on an empty cell: the row is skipped and counted
4. the database stays **under 1 MB (megabyte)**

✗ *"JARVIS shall be fast."*

Not a requirement yet: no number, no checkable condition.

✓ *"JARVIS shall answer a log command in under one second on the lab computer."* A stopwatch can decide.

User stories — As a..., I want..., so that...

As a **role**, *I want* **goal**, *so that* **benefit**.

One requirement, from the point of view of the person who benefits. The Agile Alliance dates the template to 2001. Brick 2: **five stories, at least three kinds of role**.

As the owner,

I want **a monthly cost for each routine**,

so that **I can decide what to automate first**.

As a staff member,

I want **to log a routine with one command**,

so that **recording my time takes seconds**.

As JARVIS with no API (Application Programming Interface) key,

I want **to understand requests with rules**,

so that **the assistant still works offline**.

The third role is the system itself in a degraded condition: the offline rule deserves its own story.

INVEST (Independent, Negotiable, Valuable, Estimable, Small, Testable), the checklist for stories published by Bill Wake in 2003:

- **I** Independent
- **N** Negotiable
- **V** Valuable
- **E** Estimable
- **S** Small
- **T** **Testable**

Today the last letter matters most: a story must be **testable**.

Acceptance criteria — Given, When, Then

Acceptance criteria are the conditions a story must satisfy to be accepted as finished. A format from **BDD (Behaviour-Driven Development)**, written in Cucumber's Gherkin language:

Given
the starting situation

→

When
an action happens

→

Then
an observable result follows

✓ Given an empty log, when I type `log 25 min email_triage`, then **one entry** is stored with routine `email_triage` and minutes `25`.

✓ Given an empty log, when I type `log twenty min email_triage`, then **nothing is stored** and JARVIS replies with the expected format.

One criterion = one automated test

In **Class 10** you write exactly these as **pytest** tests.

Two rules to remember

A story without acceptance criteria can never be declared finished; an **NFR** (Non-Functional Requirement) without a threshold can never be tested.

Activity — one FR and one NFR, swap and check

4 min

- On a card, write **one functional** and **one non-functional** requirement for JARVIS, using **shall**.
- **Swap** cards with your neighbour.
- Your neighbour checks three things:
  - Can it be read in only one way?
  - Does it contain only one requirement?
  - How exactly would you test it: what would you type, measure or inspect?
- An adjective such as *fast* or *simple*? Rewrite it together, **with a number and a unit**.

Card — FR (Functional Requirement)

JARVIS shall list the routines ranked by monthly cost.

Card — NFR (Non-Functional Requirement)

JARVIS shall answer a log command in under one second on the lab computer.

Example cards: one reading, one requirement, a test you can describe.



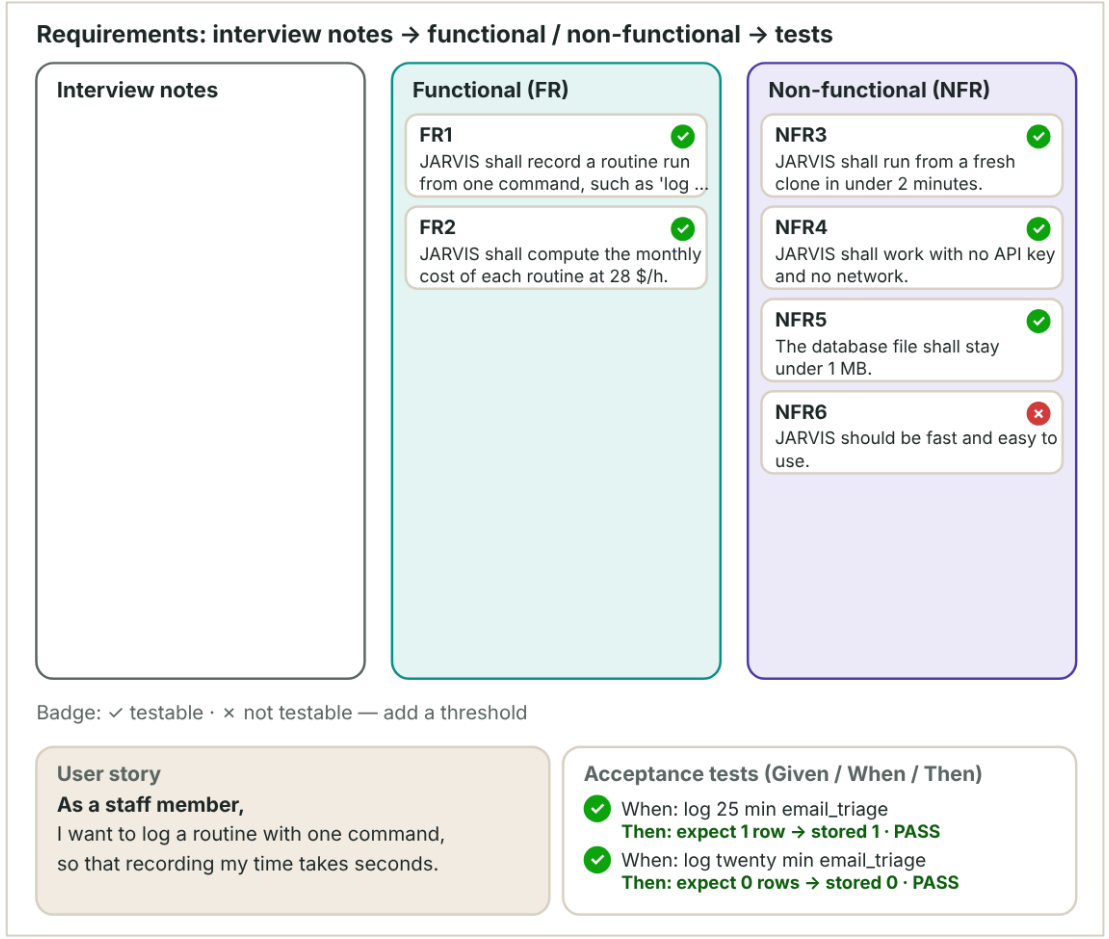
Real-life example — from a 20-minute interview to tested requirements

Six statements, as said in the interview:

- ✓ **FR1 (Functional Requirement 1)** — Record a routine run from one command.
- ✓ **FR2 (Functional Requirement 2)** — Compute the monthly cost of each routine at 28 $/h.
- ✓ **NFR3 (Non-Functional Requirement 3)** — Run from a fresh clone in under 2 minutes.
- ✓ **NFR4 (Non-Functional Requirement 4)** — Work with no **API** (Application Programming Interface) key and no network.
- ✓ **NFR5 (Non-Functional Requirement 5)** — Keep the database file under 1 **MB** (megabyte).
- ✗ **NFR6 (Non-Functional Requirement 6)** — "Should be fast and easy to use."

Then the team…

rewrites **NFR6** with thresholds (under 1 second; first routine logged within 5 minutes with the README), writes the staff member's story, and runs its two acceptance criteria on a prototype of `log`: both **PASS**. Next: Brick 2 §2, the OfflineProvider (Brick 7), pytest (Bricks 9–10).



Demo 2 — requirements classifier and acceptance checks. Each statement moves to its column with a testable badge; the two Given/When/Then criteria run as tests.

Figure 2 — the requirements map of JARVIS

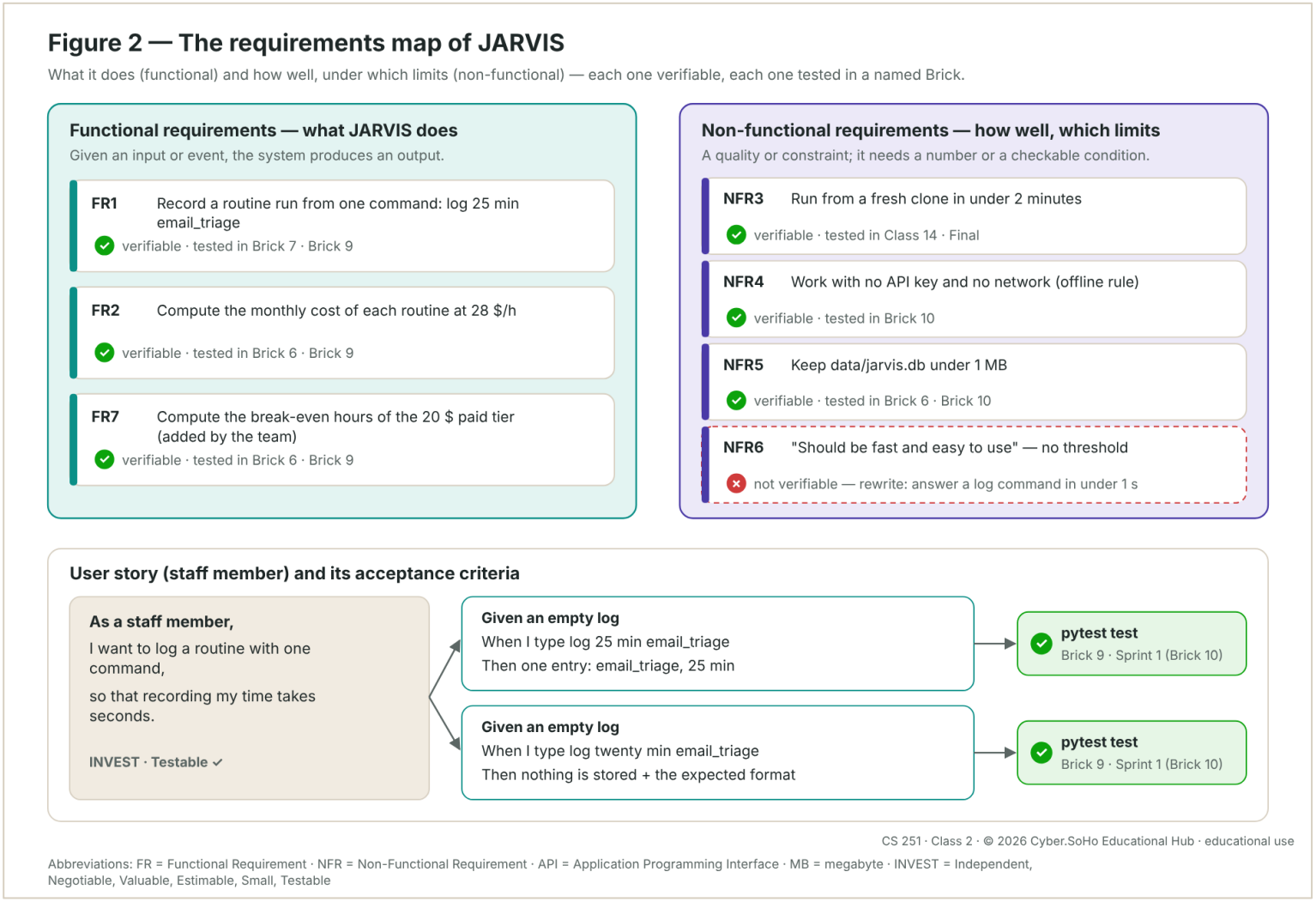


Figure 2 — The requirements map of JARVIS: functional requirements (what it does) and non-functional requirements (how well, under which limits), the user story, its Given/When/Then criteria, and the Brick in which each one is tested.

How to read it

- Left: functional requirements — what JARVIS does.
- Right: non-functional requirements — how well, under which limits; **NFR6** (Non-Functional Requirement 6) must be rewritten.
- Bottom: the staff member's story, its Given/When/Then criteria and their tests.
- Every item names the Brick in which it is tested.

Remember — Lesson 2

1	**Functional or non-functional: what is the difference?**	Functional: what the system does — a behaviour from input to output. Non-functional: how well or under which limits it does it — a quality or a constraint.
2	**When is a requirement verifiable?**	When a test, an inspection or a measurement can prove that it is met. Adjectives such as fast or easy need a number and a unit.
3	**The user-story template?**	As a <role>, I want <goal>, so that <benefit>.
4	**What do Given, When and Then describe?**	Given: the starting situation. When: the action. Then: the observable result. One criterion becomes one test.
5	**Which verb marks a binding requirement in ISO/IEC/IEEE (International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers) 29148?**	"Shall". "Should" marks a goal or a wish, which is not binding.

▶ **Watch at home:** What's the difference between Functional & Non-Functional Requirements? — Jama Software · What is Acceptance Criteria? Definition, How To, and Best Practices — ProductPlan

3

Lesson 3 of 5 · lecture 0:30–0:41 · then a fun quiz and a zen break 0:41–0:45

Scope, SMART objectives and project selection

In this lesson

- Scope and delimitations · 0:30–0:33
- Assumptions and constraints · 0:33–0:35
- **SMART** (Specific, Measurable, Achievable, Relevant, Time-bound) objectives · 0:35–0:39
- Selecting and confirming the project · 0:39–0:41

Scope and delimitations

The **scope** is the set of results a project will deliver — features, documents, data — and the work needed to deliver them, and no more, as defined by the PMI (Project Management Institute), publisher of the **PMBOK (Project Management Body of Knowledge)** Guide. The **delimitations** list what the study will **not** do (Leedy & Ormrod).

Scope — JARVIS will deliver

- the six routines of Boréal Studio and the 14-day time log
- the statistics and economics engines
- the offline provider plus one free **API** (Application Programming Interface) tier
- the tiny SQLite database and an aggregated dashboard
- the report and the defense

Delimitations — JARVIS will not

- use voice commands
- read real mailboxes or real calendars
- be a mobile application
- use any paid service
- hold data about real people other than the team's own anonymised logs

✗ **Scope creep** — features added week after week until nothing works on the day of the defense. Writing both lists is the prevention.

Assumptions and constraints

Assumption — treated as true, without proof

Written down, because if it turns out false, the results change. Ours:

- 28 $ per hour is a fair loaded cost for the studio.
- The fourteen logged days are typical working days.
- The staff logged their minutes honestly.
- The minutes saved from day 10 continue after the pilot.

Constraint — a limit the project cannot change

Respected by every objective. Ours:

- fifteen Fridays; the Product Defense on January 22, 2027
- teams of two or three; Python only
- free **API** (Application Programming Interface) tiers whose quotas change (record the number and the date)
- no personal data in any public repository
- a database file under 1 **MB** (megabyte)

? Assumptions are **tested and discussed** — in Sprint 2 (Brick 11) your recommendation lists its risks and assumptions.

! Constraints are **respected** — every objective you write must fit inside them.



SMART objectives — and the January test

An **objective** is a result the team commits to deliver by a date. **SMART (Specific, Measurable, Achievable, Relevant, Time-bound)** was first published by George T. Doran in 1981 (*Management Review*); this course uses the most common modern letters.

S — Specific
names exactly what will be delivered

M — Measurable
states a number or an observable test

A — Achievable
fits the team's hours and the allowed tools

R — Relevant
serves one of the research questions

T — Time-bound
has a date

✗ *"Make JARVIS smart."* — none of the five letters.

✓ *"Log 14 days of routines and compute the **SD** (standard deviation) of minutes per routine by October 22."* — all five.

The January test — for every objective: *can we demonstrate this at the Product Defense on January 22?* If not, the objective is outside the scope or it must become smaller. Three tools that work are worth more than ten that do not.

Selecting and confirming the project

The default data set

`boreal_routines_2026.csv`

synthetic, already anonymised, identical for every team — so the grading is comparable.

Same business problem for everyone; same rubric in both cases.

Your own real data instead? Only if all three conditions hold:

1. **Anonymised**

   no names, no message contents — only routine names, minutes and timestamps.

2. **Written permission**

   from the organization — Notice section 7.9; a relative who owns the business is not automatically the person entitled to give it.

3. **Instructor approval**

   today in the workshop, or at the latest with Brick 2.

Any condition missing → **the default data set**. See section 7.9 of the Cyber.SoHo Notice — work involving a real organization.



Activity — fun quiz, then a zen break

4 min

Hands up — four questions

1. Is *“make JARVIS smart”* **SMART** (Specific, Measurable, Achievable, Relevant, Time-bound)?
2. Is *“log 14 days of routines and compute the **SD** (standard deviation) of minutes per routine by October 22”* **SMART**?
3. Which letter fails in *“connect JARVIS to voice, calendar and e-mail by December 18”*?
4. *“The free **API** (Application Programming Interface) quota will not change before January”*: assumption or constraint?

Zen break — 2 minutes

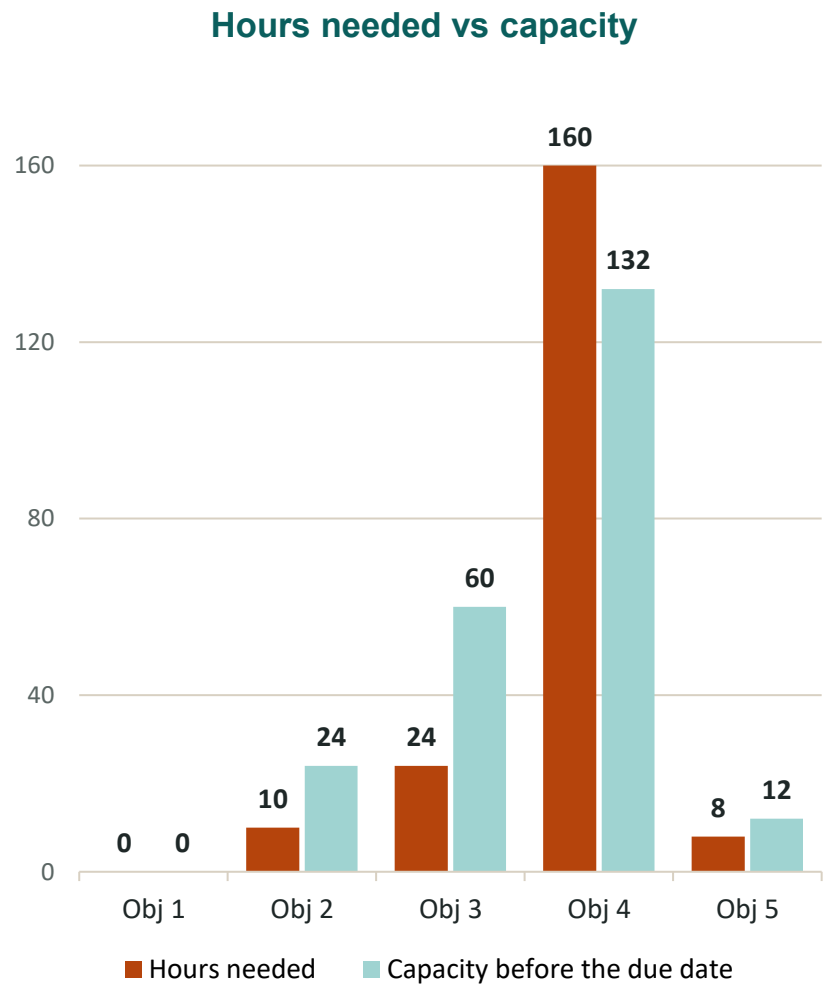
- Stand up.
- Roll your shoulders backwards five times.
- Breathe in for 4 seconds, hold for 4, breathe out for 4.
- Repeat three times.

Real-life example — five draft objectives, one January test

A team of three drafts five objectives; each one is scored on the five **SMART (Specific, Measurable, Achievable, Relevant, Time-bound)** letters and must serve **RQ1** (Research Question 1), **RQ2** (Research Question 2) or **RQ3** (Research Question 3).

#	Objective as drafted	Hours	Serves	Due	Verdict
1	Make JARVIS smart.	—	—	—	✗ rewrite
2	Log 14 days of routines; **SD** (standard deviation) of minutes per routine	10	**RQ1**	Oct 22	✓ **SMART**
3	Log e-mail triage with JARVIS offline; minutes saved per run	24	**RQ3**	Nov 6	✓ **SMART**
4	Connect JARVIS to voice, calendar and e-mail	160	**RQ3**	Dec 18	✗ rewrite
5	Break-even of the 20 $ paid tier	8	**RQ2**	Oct 15	✓ **SMART**

Capacity: 3 members × 4 h per week = **12 h per week** — 24 h before Oct 22, 60 h before Nov 6, 132 h before Dec 18. Objective 4 needs forbidden tools (a voice service, a real mailbox) and **160 h > 132 h**: it fails **Achievable** and is rewritten as Objective 3. The family-business data: no written permission, no approval → **default data set**.



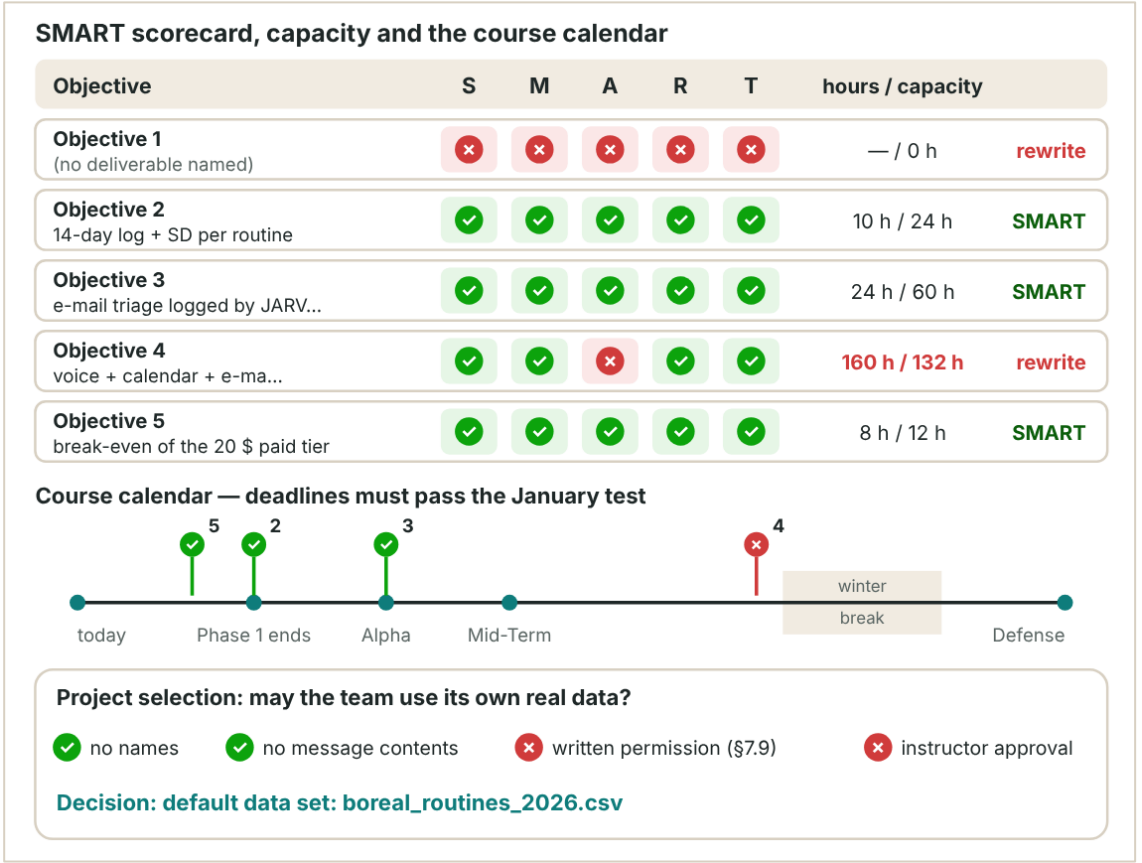
Demo 3 — the SMART scorer, statement by statement

The rule for Achievable, as the code writes it:

```
# Achievable: only allowed tools, and enough hours
tools_ok = obj["needs"] <= ALLOWED
hours_ok = obj["hours"] is not None \
           and obj["hours"] <= capacity
a = tools_ok and hours_ok

# capacity = weeks left x 12 hours per week
capacity = weeks * HOURS_PER_WEEK
```

- <= on two sets means *is a subset of*: every needed tool must be allowed.
- Objective 2: Oct 22 is 20 days away → 2 weeks → **24 h**; 10 ≤ 24 → A is true.
- Objective 4: `voice_api` is not allowed, and 160 > 132 → **only cell A turns red**.



Demo 3 — SMART (Specific, Measurable, Achievable, Relevant, Time-bound) scorer and project-selection rule (90 recorded steps). The calendar places every deadline before the January test; the data decision falls back to the default set.



Figure 3 — SMART scorecard and the course calendar

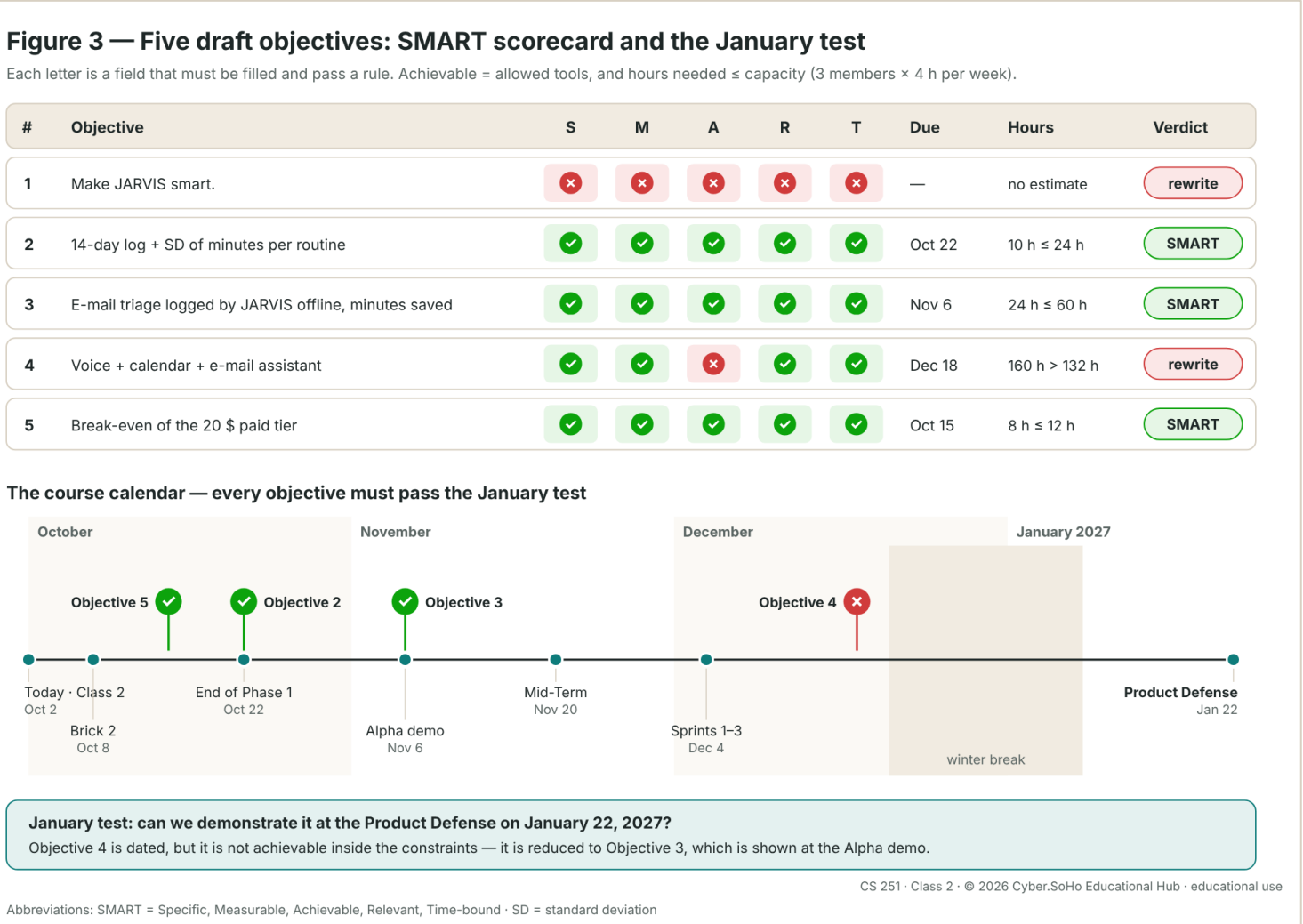


Figure 3 — Five draft objectives: SMART scorecard and the January test

Each letter is a field that must be filled and pass a rule. Achievable = allowed tools, and hours needed ≤ capacity (3 members × 4 h per week).

#	Objective	S	M	A	R	T	Due	Hours	Verdict
1	Make JARVIS smart.	✗	✗	✗	✗	✗	—	no estimate	rewrite
2	14-day log + SD of minutes per routine	✓	✓	✓	✓	✓	Oct 22	10 h ≤ 24 h	SMART
3	E-mail triage logged by JARVIS offline, minutes saved	✓	✓	✓	✓	✓	Nov 6	24 h ≤ 60 h	SMART
4	Voice + calendar + e-mail assistant	✓	✓	✗	✓	✓	Dec 18	160 h > 132 h	rewrite
5	Break-even of the 20 $ paid tier	✓	✓	✓	✓	✓	Oct 15	8 h ≤ 12 h	SMART

The course calendar — every objective must pass the January test

January test: can we demonstrate it at the Product Defense on January 22, 2027?
Objective 4 is dated, but it is not achievable inside the constraints — it is reduced to Objective 3, which is shown at the Alpha demo.

CS 251 · Class 2 · © 2026 Cyber.SoHo Educational Hub · educational use

Abbreviations: SMART = Specific, Measurable, Achievable, Relevant, Time-bound · SD = standard deviation

*Figure 3 — **SMART** (Specific, Measurable, Achievable, Relevant, Time-bound) scorecard of the five draft objectives and their deadlines on the course calendar, with the January test at the right end.*

How to read it

- Top: each objective scored on S, M, A, R and T, with its hours and capacity.
- Bottom: the course calendar from today to the Product Defense.
- Objective 4 fails Achievable: forbidden tools and 160 h > 132 h.
- Every deadline must pass the January test.



Remember — Lesson 3

1 **What does SMART stand for in this course?**

Specific, Measurable, Achievable, Relevant, Time-bound (after Doran, 1981, whose original letters were Specific, Measurable, Assignable, Realistic, Time-related).

2 **Assumption or constraint — what is the difference?**

An assumption is treated as true without proof and may turn out false; a constraint is a fixed limit the project cannot change.

3 **What is a delimitation?**

An explicit statement of what the project or study will not do — for JARVIS: no voice, no real mailboxes, no mobile app, no paid services.

4 **What is the January test?**

For every objective: can we demonstrate it at the Product Defense on January 22, 2027? If not, make it smaller or remove it.

5 **Three conditions to use your own real data?**

Anonymised (no names, no message contents), written permission from the organization (Notice section 7.9), and instructor approval — otherwise the default data set.

▶ **Watch at home:** **SMART** (Specific, Measurable, Achievable, Relevant, Time-bound) Goals: How to Set Goals That Work! (3 Minutes) — BioTech Whisperer, about 3:00 · **SMART** Goals - Quick Overview — DecisionSkills

4

Lesson 4 of 5 · lecture 0:45–0:50 · flash-cards at 0:56–1:00, with Lesson 5

References and experts

In this lesson

- Why we cite · 0:45–0:46
- Where to find credible sources · 0:46–0:48
- **APA** (American Psychological Association) 7 in one pattern · 0:48–0:49
- Finding one expert · 0:49–0:50



Why we cite — and where credible sources come from

Reference and citation

A **reference** identifies a source precisely enough for a reader to find it.

A **citation** is the short pointer in your text that sends the reader to it: (Leedy & Ormrod, 2019).

Two purposes

Verify — your reader can check the claim

Credit — you acknowledge the work you build on

In this course you cite **books and articles**, **documentation pages**, **code you reuse** and **any AI (Artificial Intelligence) assistance** — Brick rules, Notice section 18 and the University's academic-honesty policy.

Where to find credible sources

1. **Textbooks** — Leedy & Ormrod (2019); free peer-reviewed refreshers from OpenStax (Rice University).
2. **Scholarly articles** — Google Scholar and library databases; **BAnQ** (Bibliothèque et Archives nationales du Québec) is free for every Québec resident.
3. **Official documentation** — the Python documentation; the **API** (Application Programming Interface) provider's pages, read and cited with the date.
4. **Official statistics** — agencies such as Statistics Canada, for definitions and published figures.

The CRAAP test — five questions for any source

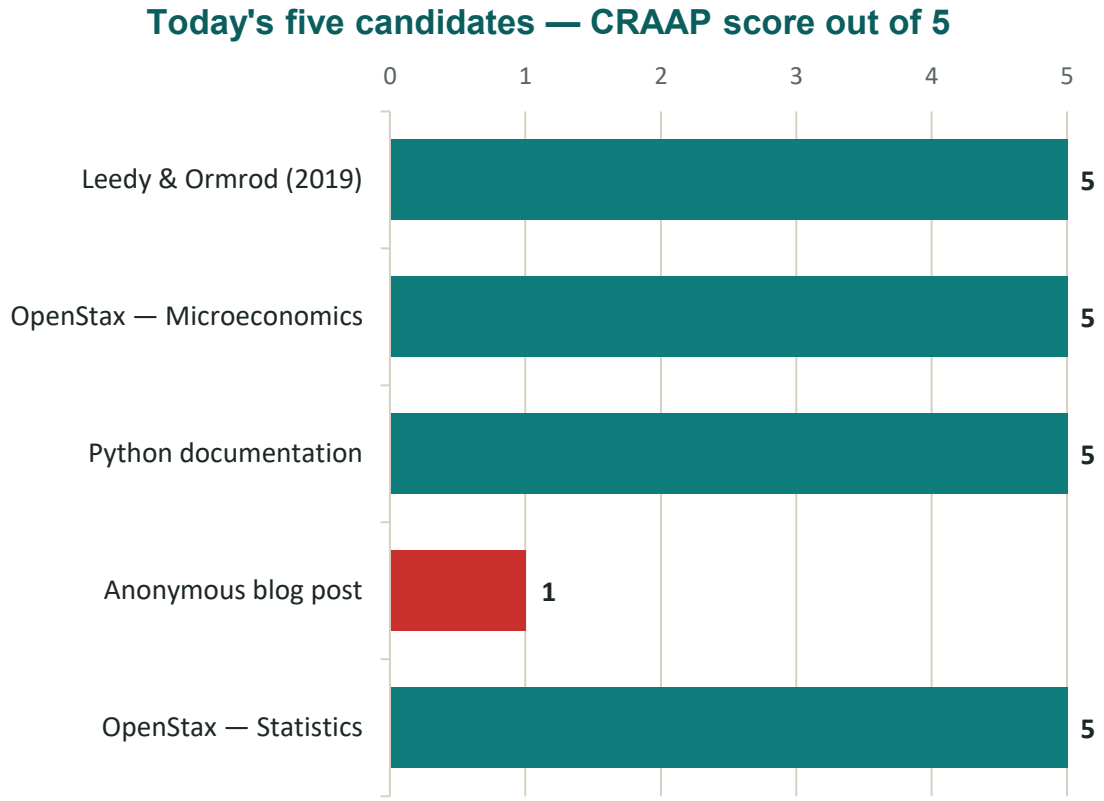
C — **Currency**
When was it published or updated?

R — **Relevance**
Does it answer my question?

A — **Authority**
Who wrote it, and what qualifies them?

A — **Accuracy**
Is there evidence? References? Can it be checked?

P — **Purpose**
Does it inform, sell or persuade?



The **CRAAP (Currency, Relevance, Authority, Accuracy, Purpose)** test was published by the librarian Sarah Blakeslee in 2004, in LOEX Quarterly, volume 31, issue 3. Wikipedia: use it to **find the primary sources** — then cite those.

The blog post passes only Relevance: no date, no identifiable author, no evidence, and it is written to sell. **Set aside.**

APA 7 in one pattern

Author. **(Date).** **Title.** **Source.**

Book

Leedy, P. D., & Ormrod, J. E. (2019). *Practical research: Planning and design* (12th ed.). Pearson Education.

Page that changes, no date

Python Software Foundation. (n.d.). *statistics — Mathematical statistics functions*. Python documentation. Retrieved October 1, 2026, from https://docs.python.org/3/library/statistics.html

AI (Artificial Intelligence) assistance

Company. (Year). *Name of the tool* (version) [Large language model]. **URL** (Uniform Resource Locator) — the pattern from **APA** (American Psychological Association)'s blog.

In the text

(Leedy & Ormrod, 2019) — or *Leedy and Ormrod (2019) state that…*; three or more authors: (Shapiro et al., 2022).

List and DOI (Digital Object Identifier)

Sorted by first author's surname; a **DOI** (`https://doi.org/...`) replaces the **URL**.



Finding one expert

An **expert**, for Brick 2, is a person with **direct experience** of the problem: a small-business owner, an office manager, a statistics tutor, an industry contact.

Your first message contains

- who you are
- the purpose
- a time limit: **20 minutes**
- three open questions, one per research question
- a request for permission to **take notes**
- how you will **anonymise** the answers (job title, not name)

RQ1 (Research Question 1)

“Which routine takes the most time in your office, and how do you know?”

RQ2 (Research Question 2)

“How do you decide whether a software subscription is worth its price?”

RQ3 (Research Question 3) — the risk

“What would make you stop using an assistant after one week?”

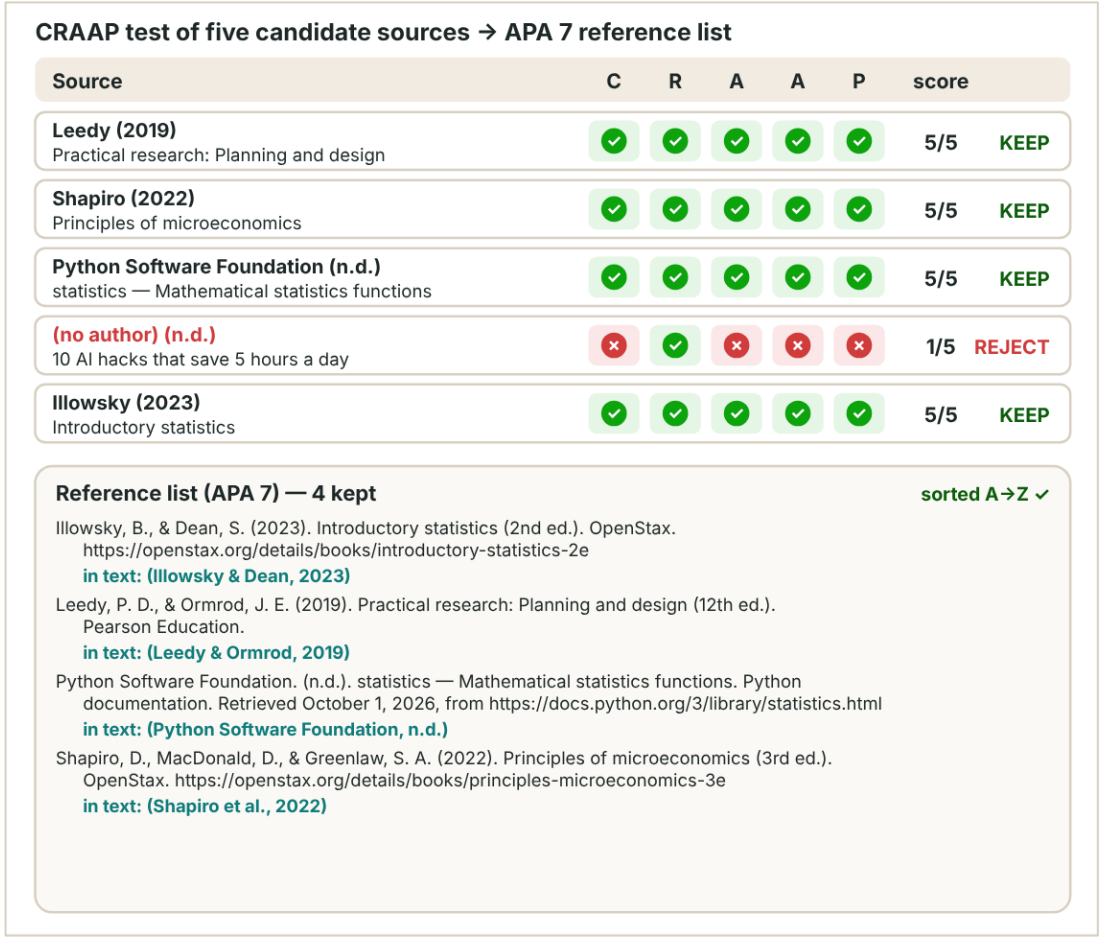


Real-life example — building the Brick 2 reference list

- **Five candidates:** Leedy & Ormrod; OpenStax *Principles of Microeconomics*; OpenStax *Introductory Statistics*; the `statistics` documentation page; a blog post *"10 **AI** (Artificial Intelligence) hacks that save 5 hours a day"*.
- **The test:** four pass all five **CRAAP** (Currency, Relevance, Authority, Accuracy, Purpose) questions; the blog passes only Relevance and is set aside.
- **The list:** four **APA** (American Psychological Association) 7 references, sorted — Illowsky & Dean (2023); Leedy & Ormrod (2019); Python Software Foundation (n.d.); Shapiro et al. (2022). A fifth is added at home.
- **The expert:** the office manager of a small accounting firm in Laval, 20 minutes, three questions, permission to take notes.

Where it goes next

Brick 2: ≥ 5 sources + expert + 3 questions. The textbooks back every formula in **Bricks 3–4**; the list grows into the References of **Bricks 9 and 12**.



Demo 4 — CRAAP evaluator and APA 7 reference builder (106 steps): five rows of five checks, then the sorted reference list with in-text citations.

Figure 4 — credible sources, CRAAP and the APA 7 reference

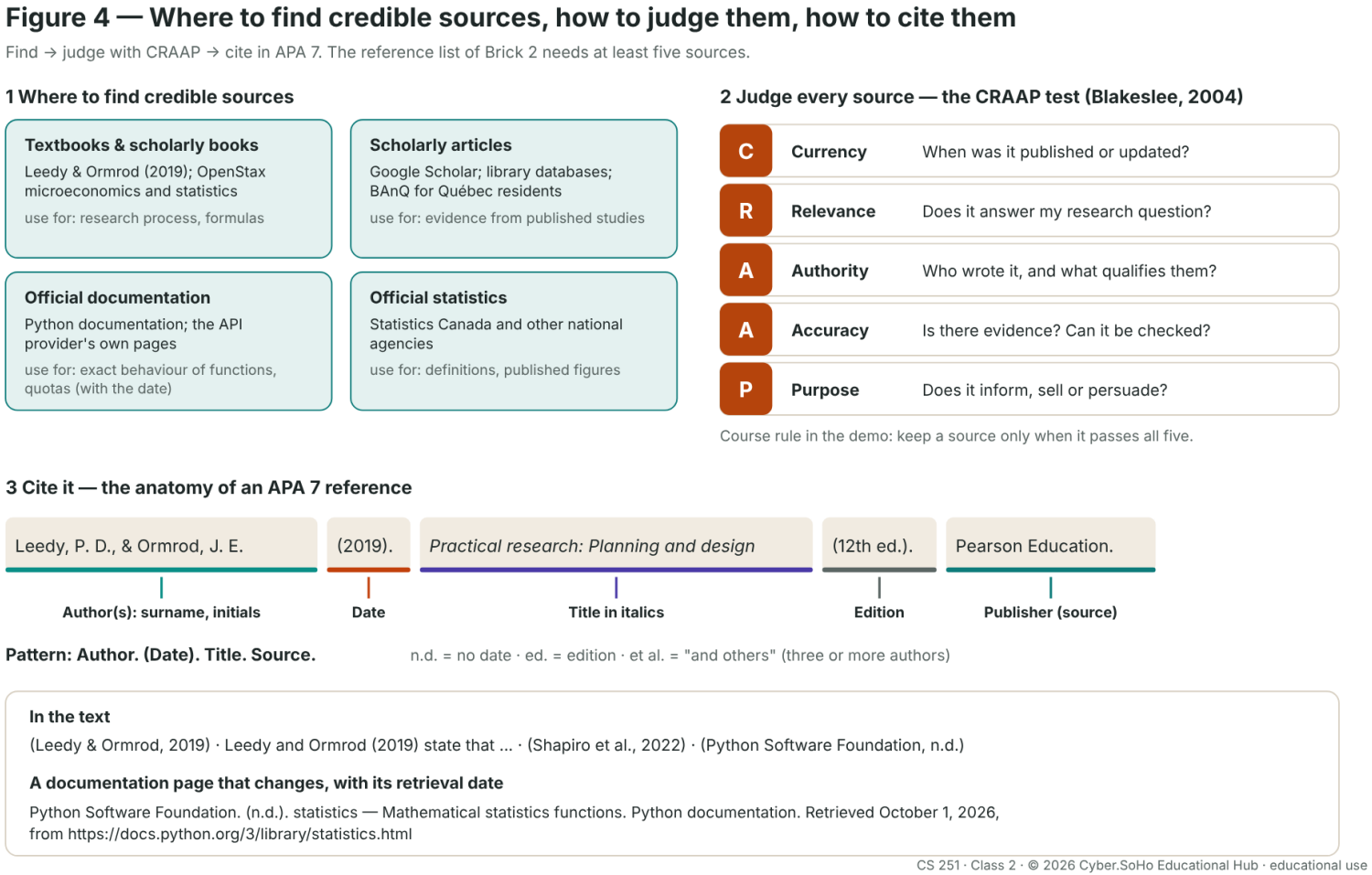


*Figure 4 — Where to find credible sources, the five **CRAAP** (Currency, Relevance, Authority, Accuracy, Purpose) questions, and the parts of an **APA** (American Psychological Association) 7 reference with its in-text citation.*

How to read it

- Left: where credible sources come from.
- Right: the five **CRAAP** questions.
- Bottom: the parts of an **APA** 7 reference and the in-text citation that points to it.

Remember — Lesson 4

1 **Reference or citation — what is the difference?**

The reference is the full entry in the list at the end; the citation is the short pointer in the text, such as (Leedy & Ormrod, 2019).

2 **What do the letters of CRAAP stand for?**

Currency, Relevance, Authority, Accuracy, Purpose.

3 **The APA (American Psychological Association) 7 pattern?**

Author. (Date). Title. Source — for a book the source is the publisher; for a web page, the site and the address.

4 **When do you add a retrieval date?**

For a page whose content is designed to change and is not archived — documentation and quota pages, for example.

5 **Besides books and articles, what must you cite in this course?**

Documentation pages, code you reuse, data sets, and any **AI** (Artificial Intelligence) assistance.

▶ **Watch at home:** The **CRAAP** (Currency, Relevance, Authority, Accuracy, Purpose) Test: Critically evaluating information sources — **QUT** (Queensland University of Technology) Library, about 2:00 · Citation: A (Very) Brief Introduction — NC State University Libraries, 1:54 · Referencing: formatting for assignments — **QUT** Library, about 3:00

5

Lesson 5 of 5 · lecture 0:50–0:56 · then flash-cards in pairs 0:56–1:00

The data dictionary and personal data

In this lesson

- What a data dictionary is · 0:50–0:51
- Python types for each column · 0:51–0:53
- What could be wrong with it · 0:53–0:55
- A time log is personal data · 0:55–0:56



What a data dictionary is

A **data dictionary** is a table that documents **every column** of a data set. It states the rules that both the people who collect the data and the code that reads it must follow.

Name	Meaning	Data type	Unit or format
`minutes`	duration of the run	`int`	minutes

Allowed values	Example	What could be wrong	
0 or more	`42`	empty; negative; 25 min	*one row of the dictionary: the column `minutes`*

Brick 2 asks for the dictionary of `boreal_routines_2026.csv` — nine columns — with one more column, **possible issue**, filled in for every row.



Python types for each column

When Python reads a **CSV** (Comma-Separated Values) file, **every value arrives as text** (`str`). The loader of Brick 5 converts each value; the **type** decides which operations are allowed — you can add two integers, not two dates.

Column	Python type	Conversion
`date`	`datetime.date`	`date.fromisoformat("2026-09-07")`
`staff_id`, `routine`, `category`, `tool`	`str` (text)	none — already text
`start_hour`, `minutes`, `interruptions`	`int` (integer)	`int("42")`
`done`	`bool`, stored as 0 / 1	`row["done"] == "1"`

✗ **Trap 1: `int("25 min")` → `ValueError`**

The unit belongs in the dictionary, not in the cell.

✗ **Trap 2: `bool("0")` → `True`**

Every non-empty text is true: never convert `done` with `bool()`.



What could be wrong — four checks of individual values

Completeness

a value is present

fails for: an empty `minutes` cell

Validity

right type, format and range

fails for: `-5` minutes · `25 min` · `08/09/2026` · start hour 27

Consistency

agrees with the agreed list and the rest of the data

fails for: `emial_triage` · `Manual` when every other row says `manual`

Uniqueness

each run is recorded once

fails for: two identical rows double the minutes

These checks become code later in the course:

Brick 5 — the loader skips and counts bad rows instead of crashing

Brick 6 — `CHECK (minutes >= 0)` and a `UNIQUE` key in the database

Brick 10 — Sprint 1 demonstrates three bad-input cases



A time log is personal data

Personal information

Information about an **identifiable individual** — **PIPEDA (Personal Information Protection and Electronic Documents Act)**, Canada's federal private-sector law, explained by the **OPC** (Office of the Privacy Commissioner of Canada). Québec's private-sector act, amended by **Law 25** and supervised by the **CAI** (Commission d'accès à l'information du Québec): information that allows a person to be identified, directly or indirectly.

De-identified — still personal

A code such as S3 replaces the name, but the data **can still be linked back**. Our staff codes are de-identification: the data stays private.

Anonymized — irreversibly

The data can **never** be linked back to a person, directly or indirectly; the operation cannot be undone.

Risk

✗ a real name where a code is expected

✗ message contents in a free-text column

✗ small groups: with five staff, a chart per person identifies the person

✗ public repositories

Mitigation

✓ codes instead of names

✓ no message contents, ever

✓ aggregated numbers per routine, never per person

✓ raw logs only in the private team repository

Activity — flash-cards in pairs

4 min

One student reads the front of a card, the other answers **before** turning it. Then swap.

Deck one — the APA (American Psychological Association) pattern

- *Author. (Date). Title. Source.*
- a book
- a documentation page with its retrieval date
- an in-text citation: two authors, three or more authors

Deck two — the Python type of every column

- `date`, `staff_id`, `routine`, `start_hour`, `minutes`, `tool`, `done`
- say the type before turning the card
- then name **one thing that could be wrong** with that column

The cards of both decks, with their answers, are the flash-cards of Lessons 4 and 5 in the lecture module.



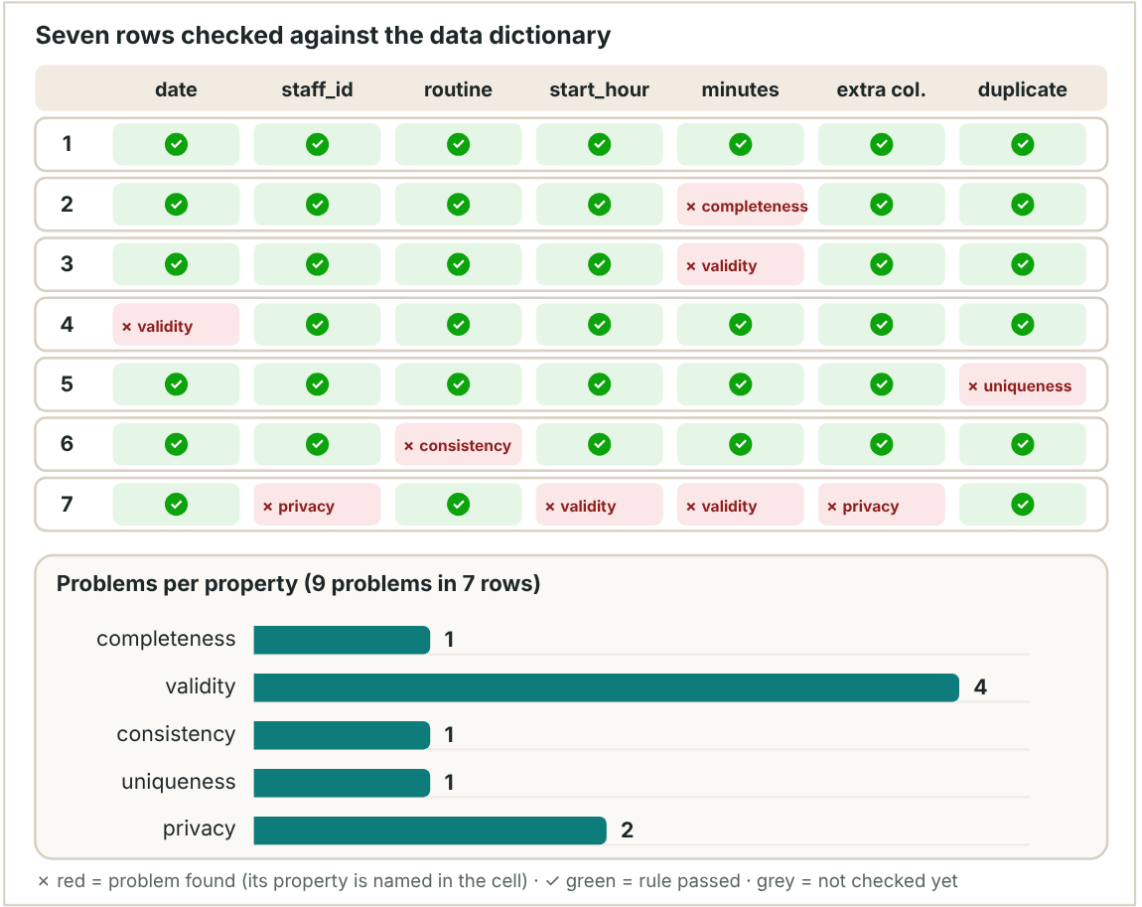
Real-life example — seven rows, one data dictionary

First Monday: the staff send seven rows by e-mail. Checked against the dictionary:

- ✓ Row 1 is correct.
- ✗ Row 2 — `minutes` empty: completeness.
- ✗ Rows 3 and 4 — `-5` minutes; `08/09/2026`: validity.
- ✗ Row 5 repeats row 1: uniqueness.
- ✗ Row 6 — `emial_triage`: consistency.
- ✗ Row 7 — `Julie Roy` instead of a code and a `note` column with message contents: **privacy** (×2); hour 27 and `25 min`: validity (×2).

Nine problems in seven rows

Rows 2–7 go back to their authors. Brick 2 gets three data-quality risks and one privacy risk with mitigations; the rules become the loader (Brick 5), the constraints (Brick 6) and the tests (Brick 10). Names are fictional.



Demo 5 — data-dictionary validator with privacy checks (233 steps): real conversions with `date.fromisoformat()` and `int()`; the bar chart counts problems per property.

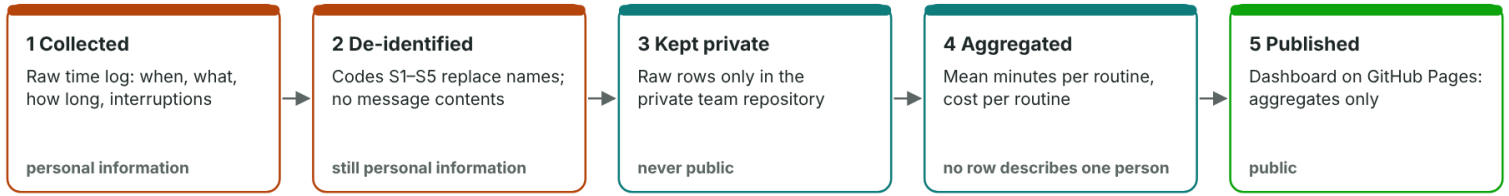
Figure 5 — the data dictionary and the privacy path

Figure 5 — The data dictionary of boreal_routines_2026.csv and the privacy path of a time log

Every value arrives as text; the dictionary says what it must become and what could be wrong. Personal data moves only along the path below.

Column	Python type	Unit / format	Allowed values	What could be wrong	Property
date	datetime.date	yyyy-mm-dd	a working day, Sept 7–24	08/09/2026 typed; missing	validity
staff_id	str	code	S1 to S5	a real name	privacy
routine	str	name	6 agreed routines	emial_triage	consistency
category	str	word	5 families	disagrees with the routine	consistency
start_hour	int	hour (24 h)	0 to 23	27	validity
minutes	int	minutes	0 or more	empty · -5 · "25 min"	completeness / validity
interruptions	int	count	0 or more	negative; decimal	validity
tool	str	word	manual, template, jarvis	Manual with a capital	consistency
done	bool (0/1)	flag	0 or 1	"yes"; bool("0") is True	validity

The privacy path of a time log (PIPEDA; Québec private-sector act as amended by Law 25)



Never: names in staff_id · message contents in a free-text column · per-person charts (five staff) · raw CSV on a public page

De-identified ≠ anonymized: a code can be linked back to a person, so the data stays private. General information, not legal advice.

CS 251 · Class 2 · © 2026 Cyber.SoHo Educational Hub · educational use

Abbreviations: CSV = Comma-Separated Values · PIPEDA = Personal Information Protection and Electronic Documents Act

Figure 5 — The data dictionary of boreal_routines_2026.csv (type, unit, rule, possible issue) and the privacy path of a time log: collected, de-identified, kept private, published only as aggregates.

How to read it

- Top: the nine columns with their Python type, unit, rule and possible issue.
- Bottom: the privacy path of a time log — collected, de-identified, kept private, published only as aggregates.
- Never: a name in staff_id, message contents, a per-person chart, a raw **CSV** (Comma-Separated Values) on a public page.

Remember — Lesson 5

1 **What does a data dictionary contain?**

For every column: name, meaning, data type, unit or format, allowed values, an example, and what could be wrong with it.

2 **Python type of date, minutes, routine and done?**

date → datetime.date; minutes → int; routine → str; done → bool, stored as 0/1 and converted with row["done"] == "1".

3 **Why does int("25 min") fail?**

int() accepts only digits with an optional sign; the text contains a unit, so Python raises ValueError. Units belong in the dictionary, not in the cell.

4 **The four checks of individual values?**

Completeness (present), validity (type, format, range), consistency (agrees with the agreed list), uniqueness (recorded once).

5 **De-identified or anonymized?**

De-identified: a code replaces the name but the data can be linked back — still personal information. Anonymized: can never be linked back, irreversibly.

▶ **Watch at home:** 5 Minute Metadata - What is a data dictionary? — Aristotle Metadata Registry, about 5:00 · Python in 100 Seconds — Fireship, 2:23 · What is personal information? — Office of the Privacy Commissioner of Canada, short (official video with transcript)

Five decisions, before any code. The workshop starts now — Brick 2.

- what we are trying to find out — problem statement and **RQ1** (Research Question 1)–**RQ3** (Research Question 3)
- what the software must do — **FR** (Functional Requirement), **NFR** (Non-Functional Requirement), user stories, acceptance criteria
- how we will know it is finished — **SMART** (Specific, Measurable, Achievable, Relevant, Time-bound) objectives and the January test
- where our evidence comes from — credible sources in **APA** (American Psychological Association) 7 and one expert
- what every column means — the data dictionary, its risks and the privacy rules

Brick 2 is due Thursday, October 8, 23:59 — submitted on time, it is part of your Mid-Term preparation already done.

© 2026 Cyber.SoHo — Educational Hub. All rights reserved. Educational content only. No affiliation with any vendor, platform or institution.

Use is subject to the Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy.

Workshop — Brick 2 “Requirements & Research Proposal v1”

One file in your team repository, `docs/requirements.md`, that turns today's five lessons into your project's **first formal document**. It will contain:

1. a problem statement (≤ 150 words) and three research questions
2. five user stories with acceptance criteria: the owner, a staff member, JARVIS with no **API** (Application Programming Interface) key
3. scope, objectives — and data approval if you propose your own data
4. ≥ 5 credible sources in **APA** (American Psychological Association) 7, one expert, three interview questions
5. the data dictionary, three data-quality risks and one privacy risk with mitigations

How the two hours run

1:00	brief
1:10	problem statement, questions, stories
1:50	data dictionary and risks
2:20	references and the expert
2:45	checkpoint: peer check with another team, then a commit

Due Thursday, October 8, 2026, 23:59 — LMS + GitHub. Brick 2 feeds sections 1–2 and the References of your report, and **Mid-Term Part B** (November 20) asks for requirements and research questions on a new scenario.



Home work (2 hours) — and next Friday

1. Finish Brick 2 and contact (or schedule) your expert.
2. Keep your 14-day time log going — it closes on Thursday, October 8.
3. Read Leedy & Ormrod, Chapter 2 (*The Problem: The Heart of the Research Process*); skim Chapter 3 (*Review of the Related Literature*) — about 50 minutes.
4. Weekly Progress Report #2: 10 minutes, with the template from the **LMS** (Learning Management System).

Thursday, October 8 · 23:59

Brick 2 due (**LMS** + GitHub).

Your **14-day time log** closes the same day.

Friday, October 9 · Class 3

Microeconomics Refresher: What Is an Hour Worth? — opportunity cost, marginal analysis, costs, break-even and payback, mapped to our columns.

RQ2 (Research Question 2) starts to receive numbers.

Where today's work goes next

Deliverable started today	Brick	Report section	Assessment link
Problem statement + **RQ1** (Research Question 1)–**RQ3** (Research Question 3)	Brick 2 (Oct 8)	§1 Introduction & Problem	Mid-Term Part B (research); Final (Methodology)
Five user stories + acceptance criteria; **FR** (Functional Requirement)/NFR list	Brick 2; tests in Bricks 9–10	§2 Requirements; §6 Testing	Mid-Term Part B (requirements); Final (Methodology, Lab Proficiency)
Scope, assumptions, constraints, **SMART** (Specific, Measurable, Achievable, Relevant, Time-bound) objectives	Brick 2; Brick 11 (risks)	§1–2; §8 Discussion	Final (Methodology, Analytical Rigor)
APA (American Psychological Association) 7 reference list + expert	Brick 2; Bricks 9 and 12	References	Final (Communication)
Data dictionary + data-quality and privacy risks	Brick 2; Bricks 5, 6, 10, 11	§3 Method; Appendix (data sheets)	Mid-Term Part B (code reading); Final (Lab Proficiency)

Nothing is thrown away: every Brick submitted on time is a page of your own Mid-Term preparation.



Links used in this lecture (1/3)

Course and legal

- Cyber.SoHo Legal Notice, Terms of Use and Acceptable Use Policy (v2.0)
- Pearson — publisher of Leedy & Ormrod, Practical Research (12th ed.)
- GitHub

Requirements

- **ISO/IEC/IEEE** (International Organization for Standardization / International Electrotechnical Commission / Institute of Electrical and Electronics Engineers) 29148:2018 — Requirements engineering
- **ISO/IEC** (International Organization for Standardization / International Electrotechnical Commission) 25010:2023 — Product quality model
- **IIBA** — International Institute of Business Analysis
- Agile Alliance glossary — User Stories
- Agile Alliance glossary — **INVEST** (Independent, Negotiable, Valuable, Estimable, Small, Testable)
- Agile Alliance glossary — Given When Then
- Cucumber — Gherkin reference

Requirements (continued)

- Mountain Goat Software — User Stories (Mike Cohn)

Scope and objectives

- **PMI** — Project Management Institute

Link safety

- VirusTotal — **URL** (Uniform Resource Locator) check
- Google Safe Browsing — site status

Sources and citation

- OpenStax (Rice University) — free peer-reviewed textbooks
- OpenStax — Principles of Microeconomics 3e
- OpenStax — Introductory Statistics 2e
- Google Scholar
- **BAnQ** — Bibliothèque et Archives nationales du Québec

Before you click: paste the address into VirusTotal. Third-party sites are outside Cyber.SoHo's control — Notice section 14 and Annex C.

Links used in this lecture (2/3)

Sources and citation (continued)

- **APA** (American Psychological Association) Style — official site (7th edition)
- **APA** Style blog — How to cite ChatGPT
- Purdue **OWL** (Online Writing Lab) — **APA** Formatting and Style Guide (7th ed.)
- Zotero — free reference manager
- **DOI** Foundation — Digital Object Identifier system
- Blakeslee (2004) — The **CRAAP** (Currency, Relevance, Authority, Accuracy, Purpose) Test, LOEX Quarterly 31(3)

Python

- Python documentation (Python Software Foundation)
- Python docs — statistics module
- Python docs — datetime (date.fromisoformat)
- Python docs — csv module

Python (continued)

- Python docs — re (regular expressions)
- Python docs — Built-in Types

Data

- Statistics Canada

Privacy

- **OPC** — Office of the Privacy Commissioner of Canada
- **OPC** — **PIPEDA** (Personal Information Protection and Electronic Documents Act), the federal private-sector privacy law
- **OPC** video — What is personal information?
- **CAI** — Commission d'accès à l'information du Québec
- Légis Québec — Act respecting the protection of personal information in the private sector (P-39.1)
- EUR-Lex — General Data Protection Regulation (EU) 2016/679

Before you click: paste the address into VirusTotal. Third-party sites are outside Cyber.SoHo's control — Notice section 14 and Annex C.



Links used in this lecture (3/3)

Videos

- NC State University Libraries — video tutorials
- **QUT** Library — Queensland University of Technology
- How to Develop a STRONG Research Question — Scribbr
- Picking Your Topic Is Research! — NC State University Libraries
- What's the difference between Functional & Non-Functional Requirements? — Jama Software
- What is Acceptance Criteria? Definition, How To, and Best Practices — ProductPlan
- **SMART** (Specific, Measurable, Achievable, Relevant, Time-bound) Goals: How to Set Goals That Work! (3 Minutes) — BioTech Whisperer
- **SMART** Goals - Quick Overview — DecisionSkills

Videos (continued)

- The **CRAAP** (Currency, Relevance, Authority, Accuracy, Purpose) Test: Critically evaluating information sources — **QUT** Library
- Citation: A (Very) Brief Introduction — NC State University Libraries
- Evaluating Sources for Credibility — NC State University Libraries
- Referencing: formatting for assignments — **QUT** Library
- 5 Minute Metadata - What is a data dictionary? — Aristotle Metadata Registry
- Python in 100 Seconds — Fireship
- What is personal information? — Office of the Privacy Commissioner of Canada

Before you click: paste the address into VirusTotal. Third-party sites are outside Cyber.SoHo's control — Notice section 14 and Annex C.